

```

int a, b, c, t;
scanf("%d%d%d", &a, &b, &c);
if(a > b) { t = a; a = b; b = t; } //执行完毕之后  $a \leq b$ 
if(a > c) { t = a; a = c; c = t; } //执行完毕之后  $a \leq c$ , 且  $a \leq b$  依然成立
if(b > c) { t = b; b = c; c = t; }
printf("%d %d %d\n", a, b, c);
return 0;
}

```

为什么这样做是对的呢？因为经过第一次检查以后，必然有 $a \leq b$ ，而第二次检查以后 $a \leq c$ 。由于第二次检查以后 a 的值不会变大，所以 $a \leq b$ 依然成立。换句话说， a 已经是 3 个数中的最小值。接下来只需检查 b 和 c 的顺序即可。值得一提的是，上面的代码把上述推理写入注释，成为程序的一部分。这不仅可以让其他用户更快地搞懂你的程序，还能帮你自己理清思路。在 C 语言中，单行注释从 “//” 开始直到行末为止；多行注释用 “/*” 和 “*/” 包围起来^①。

提示 1-20：适当在程序中编写注释不仅能让其他用户更快地搞懂你的程序，还能帮你自己理清思路。

注意上面程序中的花括号。前面讲过，if 语句中有一个“语句 1”和可选的“语句 2”，且都要以分号结尾。有一种特殊的“语句”是由花括号括起来的多条语句。这多条语句可以作为一个整体，充当 if 语句中的“语句 1”或“语句 2”，且后面不需要加分号。当然，当 if 语句的条件满足时，这些语句依然会按顺序逐条执行，和普通的顺序结构一样。

提示 1-21：可以用花括号把若干条语句组合成一个整体。这些语句仍然按顺序执行。

1.5 注解与习题

经过前几个小节的学习，相信读者已经初步了解顺序结构程序设计和分支结构程序设计的核心概念和方法，然而对这些知识进行总结，并且完成适当的练习是很必要的。

为了突出实践的重要性，本章从一开始就不加解释地给出了一段程序，并鼓励读者暂时忽略不理解的细节，把注意力集中在变量、表达式、赋值等核心内容。然而，实践的步伐也不是越快越好，因此笔者在每章的最后加入一些理论知识，供读者在实践之余稍加注意。也可以直接跳到第 2 章继续阅读，以后再阅读（并且实践）这些文字。

1.5.1 C 语言、C99、C11 及其他

本书的前 4 章介绍 C 语言，更具体地说是介绍 C99 标准中对算法竞赛而言最核心的部分。C 语言的历史和特点不难在网上以及其他书籍中找到，并且本书的前言中也详细叙述了为什么要介绍 C 语言，因此这里唯一想讲的是 C99 和编译器。

^① 单行注释原先只有 C++ 支持，后来已成为 C99 的标准的一部分。

什么是编译器？简单地说，编译器的任务就是把人类可以看懂的源代码变成机器可以直接执行的指令。“机器可以直接执行的指令”很抽象，并且笔者也无意在这里进行进一步的解释——但有一点可以说明，那就是这里的“机器”有很多种，甚至还可以是非物理的虚拟机器。诚然，让同一段程序完美地运行在千差万别的机器上并不是容易的事情，但编译器仍然大大减轻了工作量。

C 语言并不是只有一种编译器^①，例如 gcc 和微软的 Visual C++ 系列^②。为了避免同一段程序被不同的编译器编译成截然不同的机器指令，C 语言标准诞生了。目前最新的是 C11，其次是 C99。考虑到 C11 的新特性未影响算法竞赛^③，因此这里仍然讨论 C99。正如前言中所说，本书介绍 C 语言只是为学习 C++ 做铺垫。C99 中最常用的特性已经基本包含在了 C++ 中（例如 64 位整数、随处声明变量、单行注释），所以在前 4 章中无须过多地关注哪些特性是 C99 新增的，哪些是 ANSI C（即 C89）中已经包含的特性，把更多的注意力放在代码和算法本身。

本书介绍 C 语言的目的是为 C++ 语言铺垫（因为后面章节的代码用了很多 C++ 特性），但是有读者仍然希望先学习到“纯粹的 C”，所以在写作本书时确保了前 4 章中的代码全部能使用 gcc -std=c99 编译通过^④。“与 C99 兼容”是要付出代价的。例如，在 C99 中，double 的输出必须用 %f，而输入需要用 %lf，但是在 C89 和 C++ 中都不必如此——输入输出可以都用 %lf。为了保持与 C99 兼容，不得不向这种不一致性妥协。如果一开始就使用 C++，则不必拘泥于 C99，把所有代码以 .cpp 而不是 .c 为扩展名保存，用 C++ 编译器编译即可。本书前 4 章中的代码均可以直接用 C++ 编译器编译。不仅如此，多数比赛中的 C 语言都是指 ANSI C，即 C89 而不是 C99，在参加比赛时也需要把 C 语言程序当作 C++ 程序提交。

是不是很晕？没关系，只要你不是一个纯粹主义者，作者最推荐的方式就是：从现在开始直接认为你学的不是 C 语言，而是 C++ 语言中与 C 兼容的部分。这样一来，ANSI C、C99 之类的名词都和你无关了。

1.5.2 数据类型与输入格式

在继续学习之前，强烈建议读者完成以下两个实验。它们不仅能帮助你搞清楚数据类型以及输入输出的一些细节，还能培养你的实践习惯，锻炼实践能力。

数据类型实验。本章中涉及的 int 和 double 并不能保存任意的整数和浮点数。它们究竟有着怎样的限制呢？不必解释背后的原因，但需注意现象。

实验 A1：表达式 11111×11111 的值是多少？把 5 个 1 改成 6 个 1 呢？9 个 1 呢？

实验 A2：把实验 A1 中的所有数换成浮点数，结果如何？

实验 A3：表达式 $\sqrt{-10}$ 的值是多少？尝试用各种方式输出。在计算的过程中系统会报错吗？

^① 事实上，它甚至有多种解释器——无须编译直接执行的 C 语言解释器，例如 Ch 和 TCC。

^② Visual C++ 不仅包含 IDE，也包含 C 和 C++ 编译器。

^③ 有一个例外：gets 在 C11 中被移除了。详见第 3 章。

^④ 如果使用其他编译器，请自行查阅相关文档，确保代码按照 C99 标准编译，否则可能会出现编译错误。

实验 A4: 表达式 $1.0/0.0$ 、 $0.0/0.0$ 的值是多少? 尝试用各种方式输出。在计算的过程中系统会报错吗?

实验 A5: 表达式 $1/0$ 的值是多少? 在计算的过程中系统会报错吗?

输入格式实验。 本章介绍了 `scanf` 和 `printf` 这两个最常见的输入输出函数。考虑下面的函数段, 可以从实验结果总结出什么样的规律?

程序 1-15 输入输出实验

```
#include<stdio.h>
int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

实验 B1: 在同一行中输入 12 和 2, 并以空格分隔, 是否得到了预期的结果?

实验 B2: 在不同的两行中输入 12 和 2, 是否得到了预期的结果?

实验 B3: 在实验 B1 和 B2 中, 在 12 和 2 的前面和后面加入大量的空格或水平制表符 (TAB), 甚至插入一些空行。

实验 B4: 把 2 换成字符 s, 重复实验 B1~B3。

输出技巧。 读者有没有注意到在本章中所有的 `printf` 中, 双引号中的内容总是以 `\n` 结尾? `\n` 是一个特殊字符, 叫做“换行符”, 其中 `n` 是英文单词 `newline` (换行) 的首字母。换句话说, 在输出的最后加一个 `\n` 会在输出结束后换行。既然“换行”只是一个特殊字符, 完全可以用 `printf("1\n2\n");` 分两行输出 1 和 2, 并且用 `printf("1\n\n2\n");` 分三行输出 1 和 2, 并且在 1 和 2 中间换一行。更多的特殊字符将在第 3 章中介绍。但是这样一来, 问题出现了: 如果真的要输出斜线“\”和字符 `n`, 怎么办? 方法是 `printf("\\n");`, 编译器会把双斜线“\\”理解成单个字符“\”^①。

最后请读者思考这样一个问题: 如何连续输出“%”和 `d` 两个字符? 不难发现使用 `printf("%d\n");` 是不行的, 那么应该怎么办呢? 读者可以自行尝试, 也可以查阅 `printf` 的资料^②。从一开始就养成查文档的好习惯是有益的。

1.5.3 习题

程序设计是一门实践性很强的学科, 读者应在继续学习之前确保下面的题目都能做出来。请先独立完成, 如果有困难可以翻阅本书代码仓库中的答案, 但一定要再次独立完成。

习题 1-1 平均数 (average)

输入 3 个整数, 输出它们的平均值, 保留 3 位小数。

^① 这是一个很有意思的设计, 建议读者花时间琢磨一下这样做的用意。

^② 例如 <http://en.wikipedia.org/wiki/Printf>。

习题 1-2 温度 (temperature)

输入华氏温度 f ，输出对应的摄氏温度 c ，保留 3 位小数。提示： $c=5(f-32)/9$ 。

习题 1-3 连续和 (sum)

输入正整数 n ，输出 $1+2+\cdots+n$ 的值。提示：目标是解决问题，而不是练习编程。

习题 1-4 正弦和余弦 (sin 和 cos)

输入正整数 n ($n < 360$)，输出 n 度的正弦、余弦函数值。提示：使用数学函数。

习题 1-5 打折 (discount)

一件衣服 95 元，若消费满 300 元，可打八五折。输入购买衣服件数，输出需要支付的金额（单位：元），保留两位小数。

习题 1-6 三角形 (triangle)

输入三角形 3 条边的长度值（均为正整数），判断是否能为直角三角形的 3 个边长。如果可以，则输出 yes，如果不能，则输出 no。如果根本无法构成三角形，则输出 not a triangle。

习题 1-7 年份 (year)

输入年份，判断是否为闰年。如果是，则输出 yes，否则输出 no。

提示：简单地判断除以 4 的余数是不够的。

接下来的题目需要更多的思考：如何用实验方法确定以下问题的答案？注意，不要查书，也不要网上搜索答案，必须亲手尝试——实践精神是极其重要的。

问题 1: int 型整数的最小值和最大值是多少（需要精确值）？

问题 2: double 型浮点数能精确到多少位小数？或者，这个问题本身值得商榷？

问题 3: double 型浮点数最大正数值和最小正数值分别是多少（不必特别精确）？

问题 4: 逻辑运算符“&&”、“||”和“!”（表示逻辑非）的相对优先级是怎样的？也就是说， $a \&\& b \parallel c$ 应理解成 $(a \&\& b) \parallel c$ 还是 $a \&\& (b \parallel c)$ ，或者随便怎么理解都可以？

问题 5: `if(a) if(b) x++; else y++` 的确切含义是什么？这个 `else` 应和哪个 `if` 配套？有没有办法明确表达出配套方法？

1.5.4 小结

对于不少读者来说，本章的内容都是直观、容易理解的，但这并不意味着所有人都能很快地掌握所有内容。相反，一些勤于思考的人反而更容易对一些常人没有注意到的细节问题产生疑惑。对此，笔者提出如下两条建议。

一是重视实验。哪怕不理解背后的道理，至少要清楚现象。例如，读者若亲自完成了本章的探索性实验和上机练习，一定会对整数范围、浮点数范围和精度、特殊的浮点值、`scanf`、空格、TAB 和回车符的过滤、三角函数使用弧度而非角度等知识点有一定的了解。这些内容都没有必要死记硬背，但一定要学会实验的方法。这样即使编程时忘记了一些细节，手边又没有参考资料，也能轻松得出正确的结论。

二是学会模仿。本章始终没有介绍“`#include <stdio.h>`”语句的作用，但这丝毫不影响读者编写简单的程序。这看似是在鼓励读者“不求甚解”，但实为考虑到学习规律而作出的决策：初学者自学和理解能力不够，自信心也不够，不适合在动手之前被灌输大量的理

论。如果初学者在一开始就被告知“`stdio` 是 `standard I/O` 的缩写，`stdio.h` 是一个头文件，它在 `XXX` 位置，包含了 `XXX`、`XXX`、`XXX` 等类型的函数，可以方便地完成 `XXX`、`XXX`、`XXX` 的任务；但其实这个头文件只是包含了这些函数的声明，还有一些宏定义，而真正的函数定义是在库中，编译时用不上，而在连接时……”多数读者会茫然不知所云，甚至自信心会受到打击，对学习 C 语言失去兴趣。正确的处理方法是“抓住主要矛盾”——始终把学习、实验的焦点集中在最有趣的部分。如果直观地解决方案行得通，就不必追究其背后的原理。如果对一个东西不理解，就不要对其进行修改；如果非改不可，则应根据自己的直觉和猜测尝试各种改法，而不必过多地思考“为什么要这样”。

当然，这样的策略并不一定持续很久。当学生有一定的自学、研究能力之后，本书会在适当的时候解释一些重要的概念和原理，并引导学生寻找更多的资料进一步学习。要想把事情做好，必须学得透彻，但没有必要操之过急。